



Embedded System Design: Sobel Algorithm Implementation

Report

Realized by :

Oliver Daoud, 356765

Xavier Magbi , 340062

Professor

Theo Klutter

Period

April-May 2026

Introduction

The goal of this project is to accelerate an image-processing pipeline by moving computationally intensive operations away from the CPU and onto dedicated hardware. The main operation studied during the project is the Sobel filter, which detects edges in an image by calculating intensity gradients.

The project was developed incrementally. We first created a software reference implementation, then introduced a custom hardware instruction, improved memory transfers using Direct Memory Access (DMA), and finally explored optimizations based on sliding windows.

Goal

The original objective was to implement and accelerate the a movement detection algorithm that uses the Sobel edge-detection filter. The Sobel operator processes each pixel using its surrounding 3×3 neighborhood. Two kernels are applied to approximate the horizontal and vertical gradients:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

The resulting edge intensity can be approximated as:

$$G = |G_x| + |G_y|.$$

The broader goal was not only to obtain a correct output, but also to progressively reduce the workload placed on the CPU.

Software Implementation

The project began with a complete software implementation of the Sobel algorithm and movement detection. This implementation ran entirely on the CPU and served as a reference model. For each output pixel, the processor loaded the required neighboring pixels, applied the Sobel kernels, calculated the gradient magnitude, calculated the difference with the previous frame and stored the results.

At this stage, the priority was correctness. The software version made it possible to verify the expected output and establish a baseline for the following implementations. This first version required approximately 330 million cycles, with around 265 million stall cycles.

The main bottlenecks were memory accesses and CPU workload. The processor was responsible for both retrieving the pixels and performing every arithmetic operation. The next step was therefore to move the Sobel calculation itself into dedicated hardware using a custom instruction.

Simple Sobel Custom Instruction

After validating the software version, we implemented a simple Sobel custom instruction, referred to as `sobelCi`. In this first hardware-assisted version, the pixels were still fetched individually by the CPU, but the Sobel calculation was performed by dedicated hardware.

This reduced the execution time from approximately 330 million cycles to approximately 270 million cycles. The result confirmed that moving the arithmetic operations into hardware was useful, but the improvement remained limited.

The new bottleneck was the transfer of data. The CPU still needed to retrieve and forward the pixels one by one. To reduce this overhead, the following implementation introduced DMA-based transfers.

Grayscale Streaming

An important number of cycles in the software-only implementation is the grayscale conversion. Consequently, we used the modified camera module from PW6 to write the pixels in the RAM directly in grayscale format.

DMA-Based Data Transfers and Ring Buffer

To reduce CPU stall cycles and minimize CPU-managed memory operations, image transfers are performed using a DMA controller, following a similar approach as PW6. Instead of having the CPU fetch and forward every pixel individually, the DMA controller transfers image data directly from main memory into the Custom Instruction (CI) attached memory. This offloads memory traffic from the processor and reduces the number of instructions required for data movement. This CI attached memory with integrated DMA controller is implemented in `ramDmaCi.v` and `DMA.v`. It is adapted from our implementation of PW4 but with more memory meaning the control bits on `valueA` shifted one bit to the left.

To efficiently supply the Sobel operator with its required (3×3) neighborhood, four address registers are used as a ring buffer: `line1Addr`, `line2Addr`, `line3Addr`, and `line4Addr`. These registers reference four line-sized regions in the CI memory.

The Sobel filter requires data from three consecutive image lines. When processing moves from one image row to the next, two of these three lines are reused. Therefore, while the Sobel computation is performed on the current set of lines, the DMA controller simultaneously transfers the next required image line into the unused memory region referenced by `line4Addr`. This overlaps computation and data transfer, hiding most of the memory latency.

Once a line has been processed, the oldest image line is no longer required for future Sobel windows. Its memory region can therefore be reused to store the Sobel output. After the complete line has been computed, a single DMA transfer copies the resulting Sobel line back to main memory, avoiding a separate transfer for every pixel.

The ring buffer is updated by rotating the address registers. When the processing window advances by one image row, the oldest buffer becomes the free buffer for the next DMA transfer:

```

status ← line1Addr,
line1Addr ← line2Addr,
line2Addr ← line3Addr,
line3Addr ← line4Addr,
line4Addr ← status.

```

This mechanism allows the four memory regions to be reused continuously while maintaining the three-line sliding window required by the Sobel filter.

At this stage, the remaining bottleneck was data reuse. Neighboring Sobel calculations require many of the same pixels. Loading complete windows repeatedly would therefore waste memory bandwidth. To address this issue, the next optimization introduced a sliding-window buffer for efficient computation.

Sliding-Window Processing

To further reduce memory traffic, a second custom instruction, implemented in `quadSobelCi.v`, performs four Sobel computations simultaneously. Instead of processing a single 3×3 neighborhood, it maintains a 3×6 sliding window containing six consecutive pixel columns from three image rows:

$$W = \begin{bmatrix} p_{0,0} & p_{0,1} & p_{0,2} & p_{0,3} & p_{0,4} & p_{0,5} \\ p_{1,0} & p_{1,1} & p_{1,2} & p_{1,3} & p_{1,4} & p_{1,5} \\ p_{2,0} & p_{2,1} & p_{2,2} & p_{2,3} & p_{2,4} & p_{2,5} \end{bmatrix}.$$

A 3×6 window contains four overlapping 3×3 neighborhoods. Therefore, a single window can produce four Sobel magnitudes in parallel:

$$\begin{aligned}
W_0 &= \begin{bmatrix} p_{0,0} & p_{0,1} & p_{0,2} \\ p_{1,0} & p_{1,1} & p_{1,2} \\ p_{2,0} & p_{2,1} & p_{2,2} \end{bmatrix} & W_1 &= \begin{bmatrix} p_{0,1} & p_{0,2} & p_{0,3} \\ p_{1,1} & p_{1,2} & p_{1,3} \\ p_{2,1} & p_{2,2} & p_{2,3} \end{bmatrix} \\
W_2 &= \begin{bmatrix} p_{0,2} & p_{0,3} & p_{0,4} \\ p_{1,2} & p_{1,3} & p_{1,4} \\ p_{2,2} & p_{2,3} & p_{2,4} \end{bmatrix} & W_3 &= \begin{bmatrix} p_{0,3} & p_{0,4} & p_{0,5} \\ p_{1,3} & p_{1,4} & p_{1,5} \\ p_{2,3} & p_{2,4} & p_{2,5} \end{bmatrix}
\end{aligned}$$

The four resulting 8-bit Sobel values are packed into a single 32-bit word, allowing the processor to write four output pixels to the CI memory with a single memory operation.

The input pixels are first stored in the custom-instruction attached memory using DMA transfers. The processor then reads one 32-bit word from each of the three image rows. Since each word contains four grayscale pixels, the processor extracts the corresponding pixel columns and sends them to the `quadSobel` custom instruction. Each transfer therefore provides one complete column of three pixels:

$$\begin{bmatrix} p_{0,i} \\ p_{1,i} \\ p_{2,i} \end{bmatrix}.$$

After the initial six columns have been loaded, the Sobel unit can produce four output pixels. The window is then advanced by four pixel positions. Rather than reloading an entirely new 3×6 window, the two rightmost columns are retained while only four new columns are inserted. Consequently, 12 new pixels are transferred instead of the 18 pixels that would be required to rebuild the complete window.

This organization exploits the strong overlap between adjacent Sobel neighborhoods and significantly reduces both memory accesses and custom instruction invocations. Combined with the DMA-based line buffering described previously, it reduced the execution time from approximately 270 million cycles for the software implementation to approximately 71 million cycles per frame.

Frame Differencing

After implementing the Sobel-related improvements, the pipeline was completed with frame differencing. This operation compares two consecutive frames, F_t and F_{t+1} :

$$D_t(x, y) = |F_{t+1}(x, y) - F_t(x, y)|.$$

The result identifies pixels whose intensity changed between the two frames. This can be used as a basic form of motion detection.

As the main bottleneck was the Sobel computation, the frame-difference stage was implemented in software. Memory space was allocated to store the previous frame. After computing a new 32-bit Sobel word (containing four pixels), the corresponding word from the previous frame was loaded, the absolute difference was computed, and the result was written to the custom-instruction memory for subsequent DMA transfer. The previous-frame buffer was then updated with the current Sobel word for use during the next frame.

To reduce the influence of noise, a simple magnitude filter was applied to the Sobel output before frame differencing. The five least-significant bits of each pixel were cleared using the mask

`sobel & ~0x1F1F1F1F`

thereby suppressing small gradient values while preserving stronger edges.

Current and Future Work

The current optimization uses both custom-instruction inputs to transfer image data. Previously, `valueB` loaded one new column while `valueA` was used as a flag. In the next version, `valueA` and `valueB` will each provide one column:

```
col0 ← col2,    col1 ← col3,  
col2 ← col4,    col3 ← col5,  
col4 ← valueA,  col5 ← valueB.
```

This doubles the amount of data passed to the custom instruction per call. After preserving two columns from the previous window, only two calls are needed to load the four new columns required for the next 3×6 window, instead of four calls previously.

A more radical approach would be to avoid transfer from one CI memory to another by implementing the sliding window inside the CI memory with the DMA. This could lower even more the cycle count.

Another consideration to accelerate the program would be to start the DMA transfer from the CI memory to the frame buffer before the computation of all pixels has ended. If handled carefully it could allow for more transfer and calculation overlap.

Reduction of Volatile Accesses

Another performance improvement was obtained by reducing the number of variables declared with the `volatile` qualifier. This qualifier is required for memory-mapped hardware registers, but temporary values used only for local calculations do not need to be volatile.

Unnecessary uses of `volatile` can prevent the compiler from keeping values in processor registers and may force additional memory accesses. By using normal variables where possible, the execution time decreased from approximately 71 million cycles to approximately 29 million cycles. With a clock frequency of 70MHz this corresponds to 2.6 Frames/s

Conclusion

The project started with a CPU-based Sobel implementation and progressively introduced hardware acceleration. Each stage addressed a limitation of the previous version.

The simple custom instruction accelerated the calculation itself, but still relied on pixel-by-pixel transfers managed by the CPU. DMA transfers reduced this dependency. The sliding-window optimization then improved data reuse by avoiding unnecessary memory accesses. Finally, frame differencing extended the system toward motion detection.